WORKING IN A LATTICE DIAMOND ENVIRONMENT

Content

1 Creating a new project 2

2 Creating a diagram 9

3 Developing a modelling project 16

4 Test construction 18

5 Modelling the design scheme 24

Lattice Semiconductor's electronic circuit design tools support all stages of the design process, from drawing or describing schematics in high-level hardware description languages VHDL and Verilog, to chip programming, including debugging of the designed hardware. Their use in laboratory work is illustrated in Figure 1:

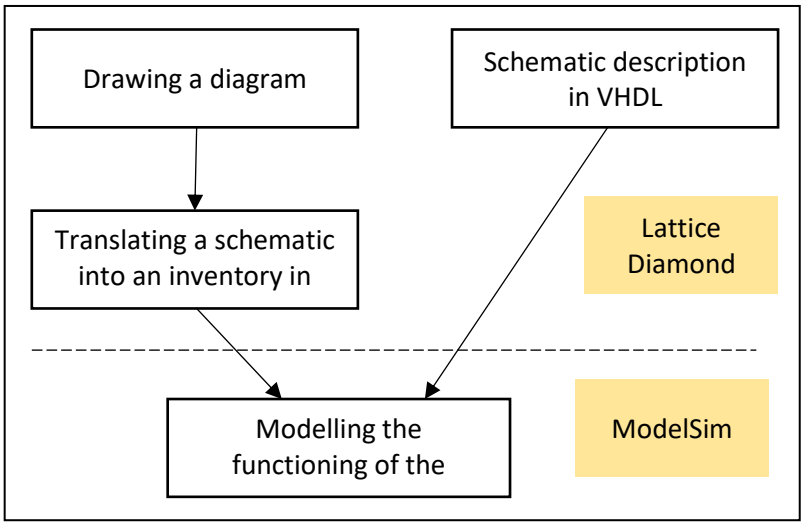


Figure 1: Using Lattice Diamond tools

This inventory is geared towards laboratory work, so only information on tools that will be needed for laboratory work is provided.

1. Developing a new project

Lattice Diamond uses the project concept. A project consists of files that schematically or logically describe the objects and device types of the project, also known as *source* files, as well as intermediate data and output data (results) files, which are stored in a single directory (folder).

When creating a project, it is recommended that you start by creating a new folder and giving it a name for the lab you are working on (e.g. labor1).

The Lattice Diamond system starts. When the system is started, the *Start Page* window opens (Figure 2).

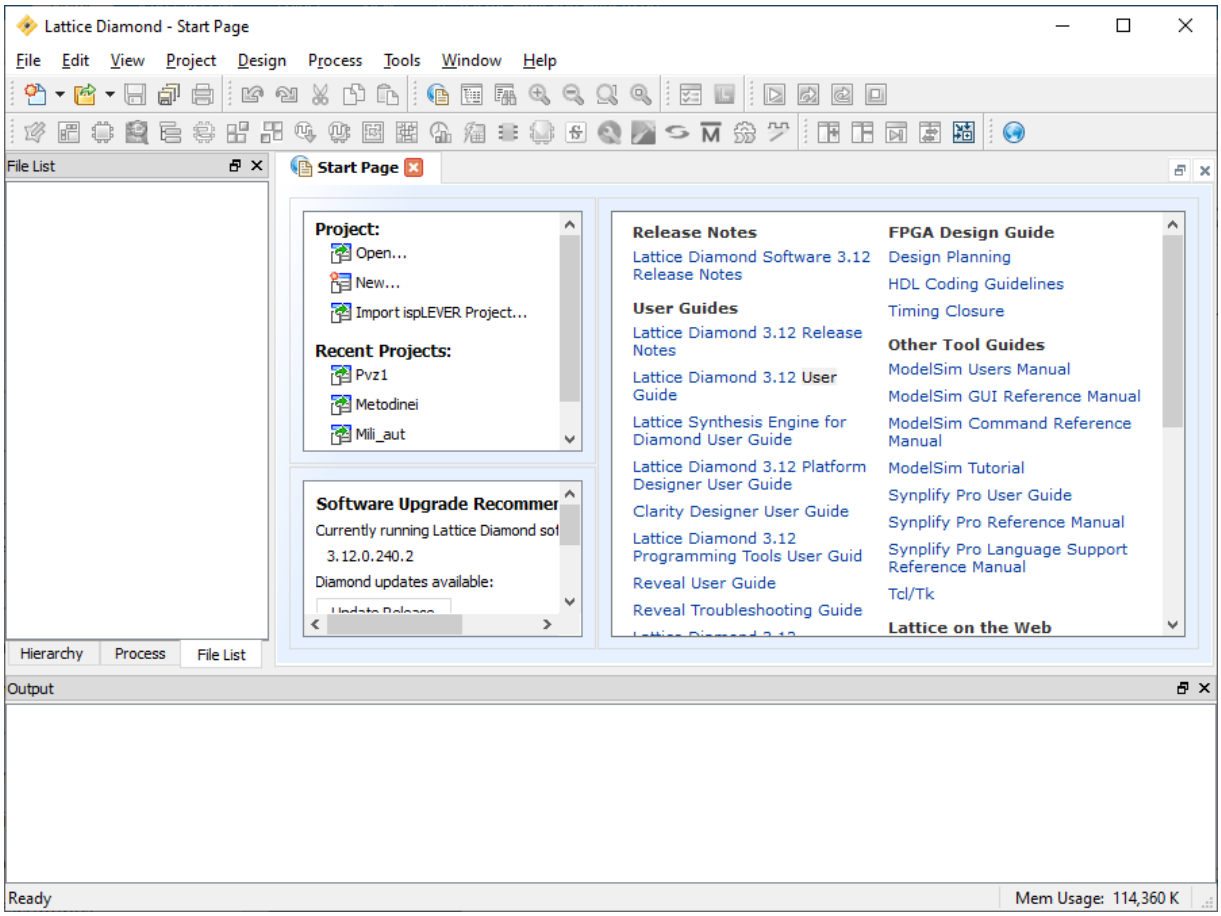


Figure 2: Lattice Diamond home window

In the Start Page window, select **New** in the *Project* box. This will open the Start Page of the Project Wizard. Clicking on the **Next** button will open the New Project window, shown in Figure 3.

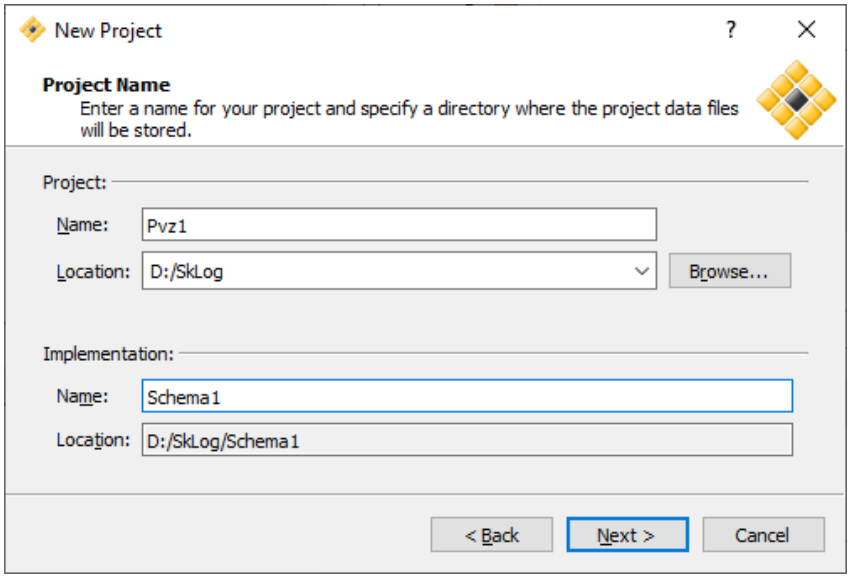


Figure 3: Project name window

In the Name box of the window that appears, enter the project name (in the figure, the project name is Ex1) and in the *Location* box, specify the location where the project files will be stored. Remember this location, because the next time you resume your work, you will find the results of your previous work there.

Then click the **Next** button. Leave the Additional Project Sources window empty. Once we have created the schematic, we can upload it next time by clicking the **Add Source** button.

Clicking on the **Next** button opens the Chip Parameters selection window shown in Figure 4.

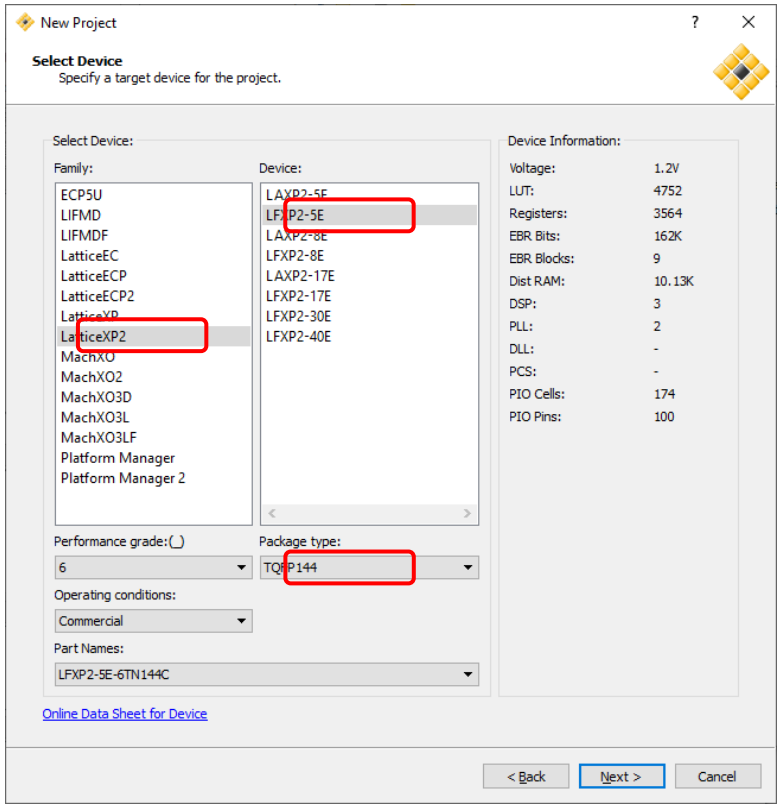


Figure 4. Microchip parameters selection window

In the boxes you must select:

- *Family - LatticeXP2,*
- *Device - LFXP2-5E,*
- *Package type - TQFP144,*
- *Performance grade - 6*
- *Operating conditions - Commercial*.

Click the **Next** button. In the modelling tool window that appears, select ***Synplify Pro*** and click the **Next** button.

The project information window shown in Figure 5 opens.

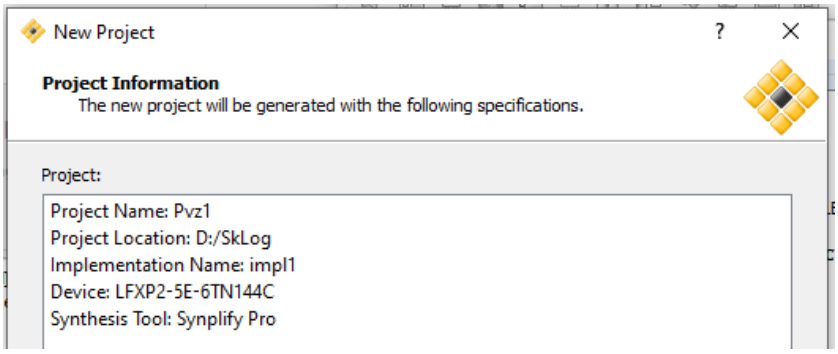


Figure 5: Project information window

Click the **Finish** button. The Project Reports window, shown in Figure 6, opens.

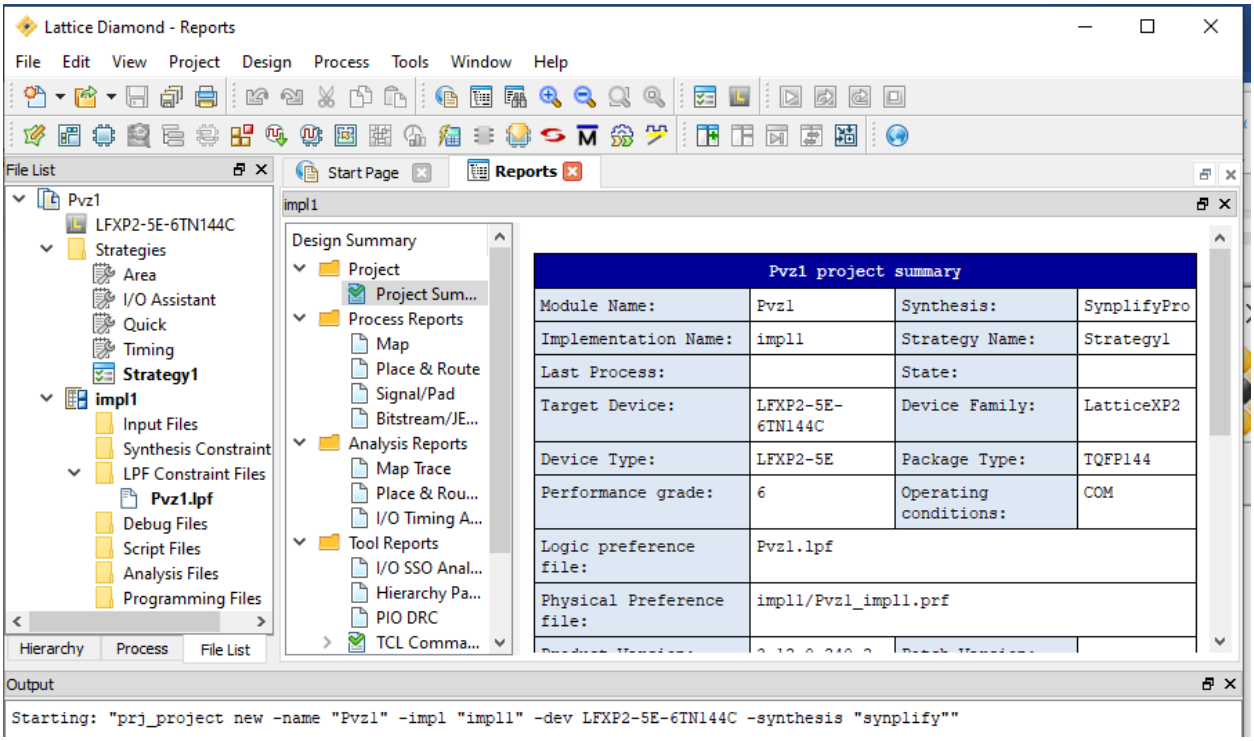


Figure 6: Project reports window

In the **File List** tab, right-click on the project name (Example1). In the drop-down window that appears (Figure 7), select ***Properties***. The Project Properties window, shown in Figure 8, appears.

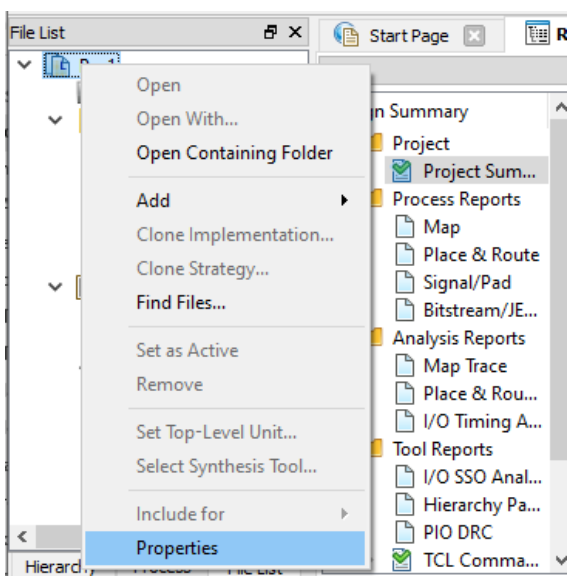


Figure 7: Drop-down window in the Project Reports window

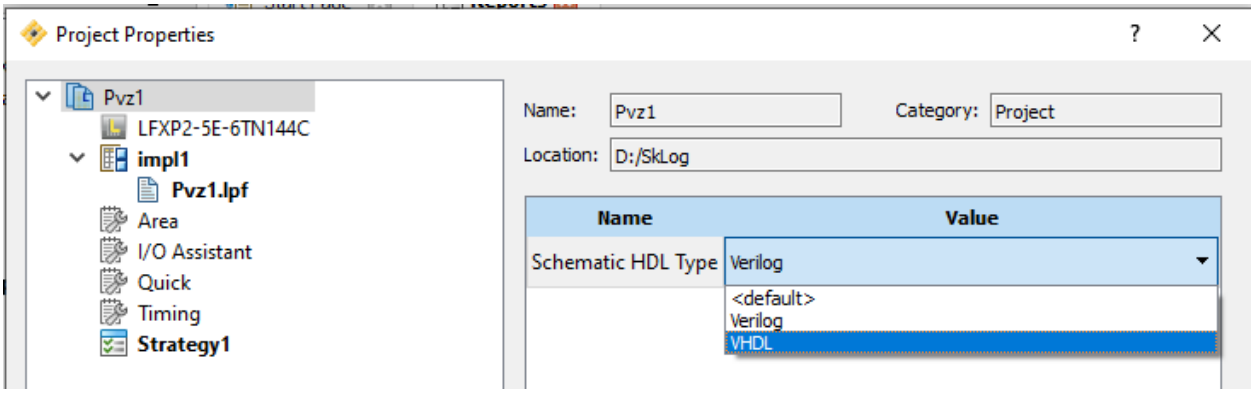


Figure 8: Project properties window

In the *Value* box of the project properties window that appears, we find ***Verilog***. The VHDL (*Very High Speed Intergrated Circuit Description Language*) language will be used in the digital logic lab work. For this reason it is necessary to change the selection to ***VHDL***.

Click **OK** to return to the Project Reports window. In the **File List** tab of this window, right-click ***Input Files in the*** second level menu. In the drop-down window as shown in Figure 9, select ***Add*** and in the next drop-down window select ***New File***.

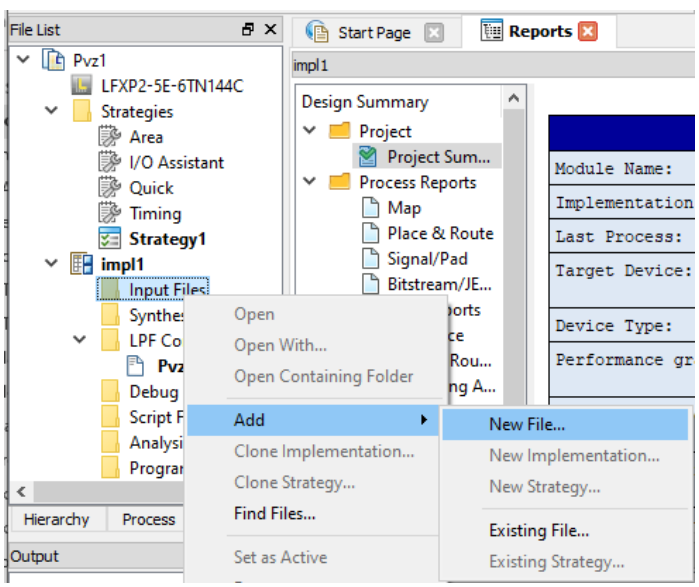


Figure 9. Uploading a new file to a project

In the New *File* window, shown in Figure 10, select ***Schematic Files*** in the *Source Files* box and write the name of the new file in the *Name* box. The file will be saved to the location specified in the *Location* box. The location can be changed by clicking **Browse**... After selecting the folder, click **Select Folder**. The *Add to Implementation* checkbox must be ticked.Then click **New** in the new file window (Figure 10).

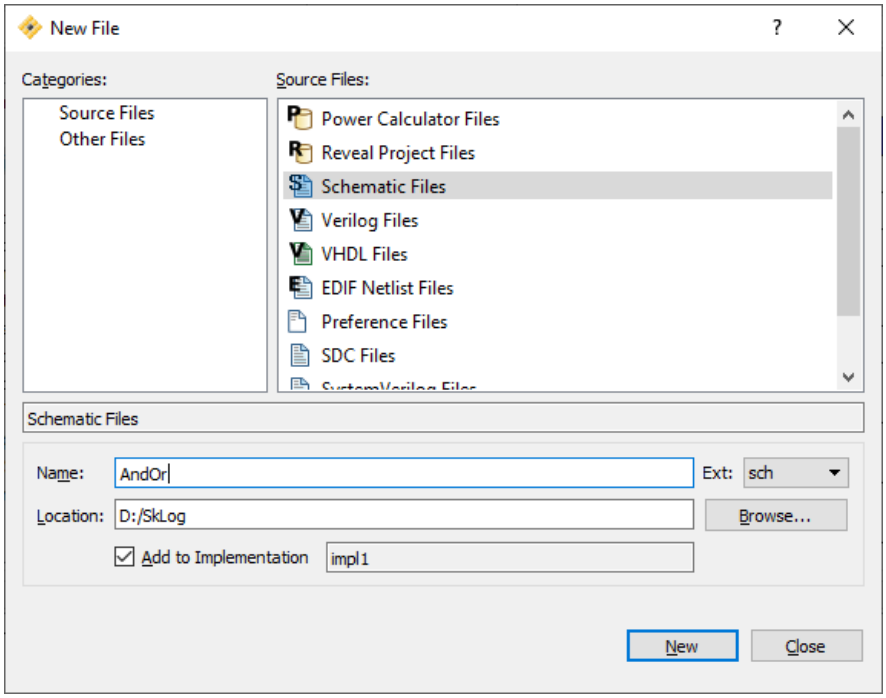


Figure 10. New file creation window

2. Creating a scheme

The Schematic Editor tab opens in the project's main window (Figure 11).

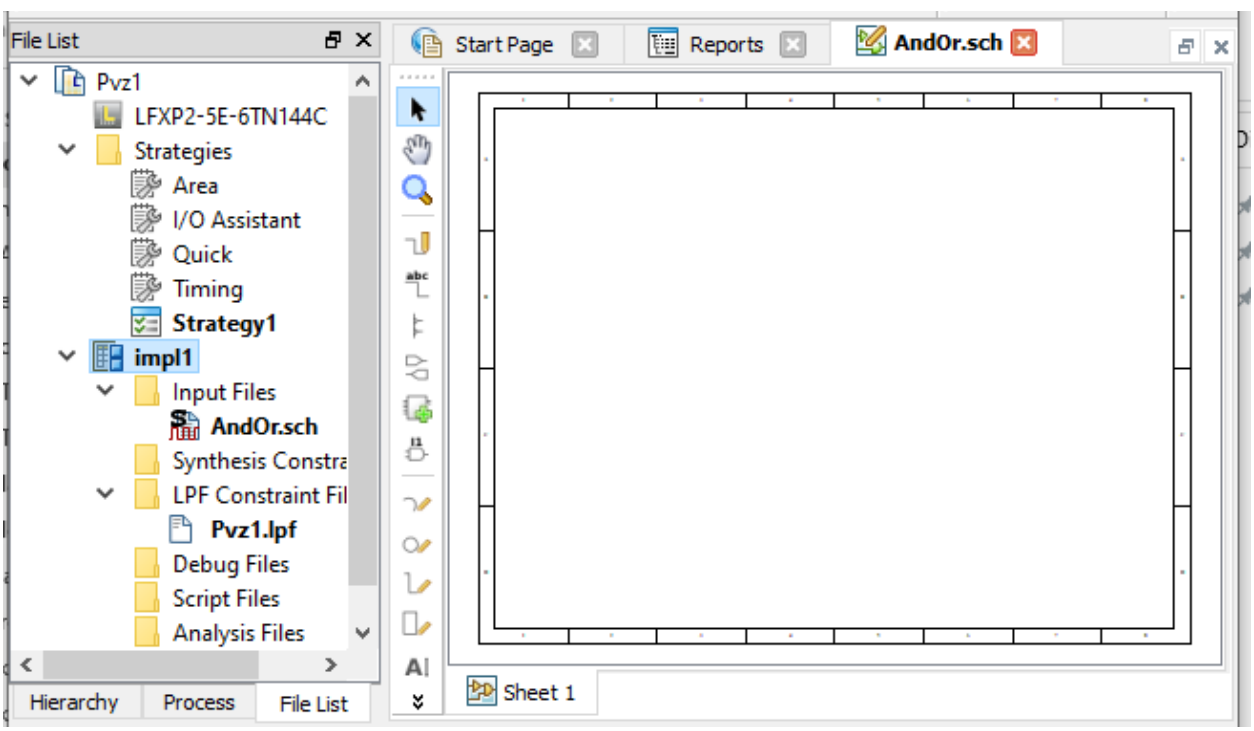


Figure 11: Schematic editor

In the Schematic Editor tab, the sheet on which the schematic will be drawn appears. If the size of this sheet is not satisfactory, it can be changed by selecting a different window size from the list of available sizes in the main menu item **Edit→Sheet**.

For plotting a diagram, it is convenient to have a grid visible in the plotting window. It can only be seen when the window is enlarged quite significantly (Figure 12):

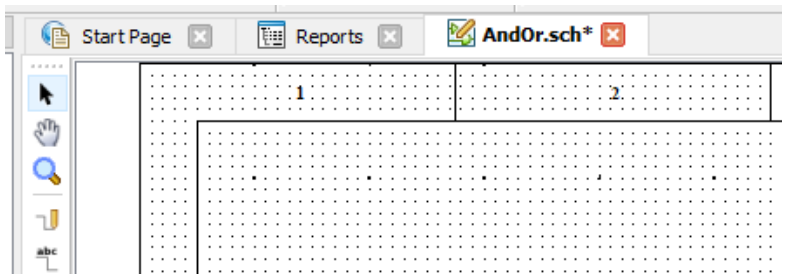


Figure 12: Drawing window grid

If there is no grid, it can be added by selecting **Tools→Options** from the main menu. In the window that appears, choose **Schematic Editor→Graphics→Show Grid**.

Once the environment is properly arranged, we start drawing the scheme. At the beginning, we load the logical elements. This can be done in two ways:

1) Right-click on the schematic sheet and select **Add→ Symbol** from the context menu (logical elements and other schematics are called *Symbols* in this design system). The *Add Symbol* window, shown in Figure 13, will open.

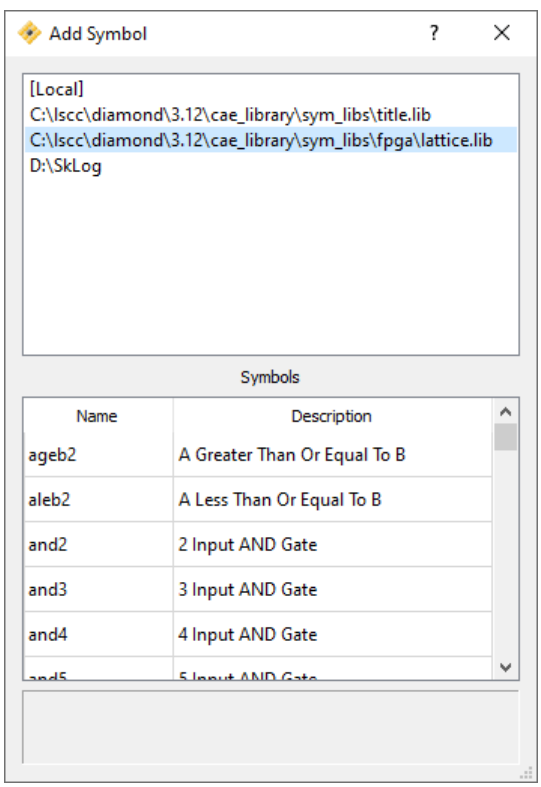


Figure 13. Add graphic symbols window

Libraries are shown in the top part of the *Add Symbol* window. For this lab we will use the C:\lscc\diamond\3.12\cae_library\sym_libs\fpga\lattice.lib library. The *Symbols* field displays the elements in the library. The desired element is selected by left-clicking on it.

2) *The Add Symbol* window can also be opened in a different way by clicking on the tool in the vertical toolbar on the left of the drawing window.

When you move the mouse cursor on the sheet, a graphic marker (symbol) of the selected item appears next to the cursor. By pressing the left key, the element is placed on the sheet, the graphic symbol of the element appears again next to the cursor, and the left key can be used to place the same element in a different position, as shown in Figure 14. When you press <Esc> or the right mouse button, the graphical marker of the element disappears at the mouse cursor (if you select another element, the old marker is replaced by the new one without pressing the right mouse button).

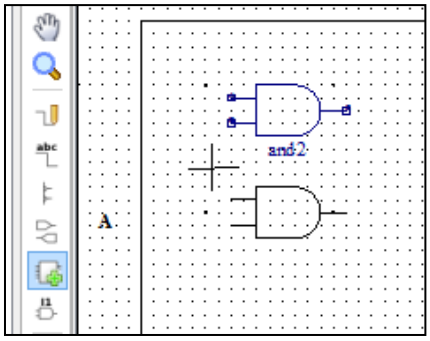


Figure 14: Adding logic elements to a circuit

The elements shown in Figure 14 are oriented horizontally (all elements are initially oriented horizontally). To rotate an element, the <Ctrl>+<R?

Once the elements have been added, they need to be connected to each other and to the input variables and the circuit output. To do this, right-click over a free space in the schematic and select **Add→Wire** (or the tool in the vertical toolbar of the schematic editor).

The purpose of the tools is shown in Figure 15.

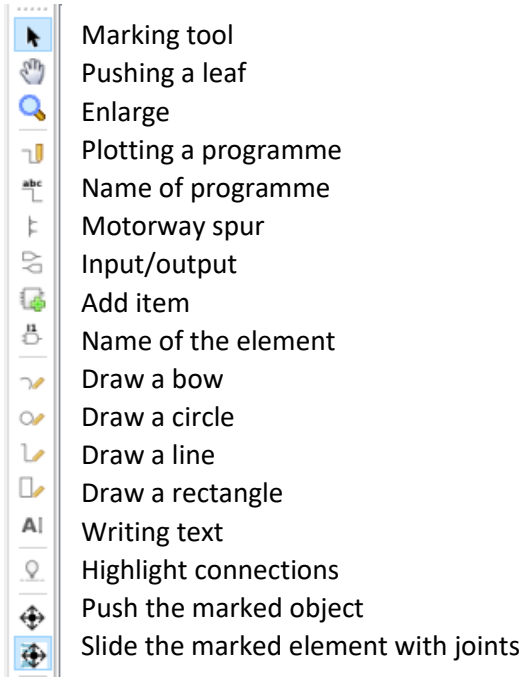


Figure 15: Vertical toolbar of the schematic editor

Click on the fourth icon from the top in the toolbar (Drawing a cord). Then move the mouse cursor to the position where the start of the merge line will be, press the left mouse button and move the mouse cursor to the position where the end of the merge line will be, and release the left mouse button. Only vertical and horizontal lines may be drawn. If the lines intersect, a rectangle indicates the connection ("contact"). If there is no rectangle at the point of intersection, the lines do not connect. To avoid errors when drawing the diagram, it is necessary to pay attention to this.

As an example, we will design a single-core adder, the principles of which you already know (from the ***Introduction to Computer Science*** course). The correctness table describing the adder's operation is shown in Figure 16.

a	b	c	P	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

Figure 16: Correctness table describing the work of the adder

A transfer **P** is formed when at least 2 of the inputs are equal to 1:

$$P = a.b + a.c + b.c.$$

The formation of the sum **S** is described by the following function:

$$\mathbf{S = a.b.\overline{c} + a.\overline{b}.c + \overline{a}.b.c + \overline{a}.b.c.}$$

A schematic with the logic elements and interconnections is shown in Figure 17.

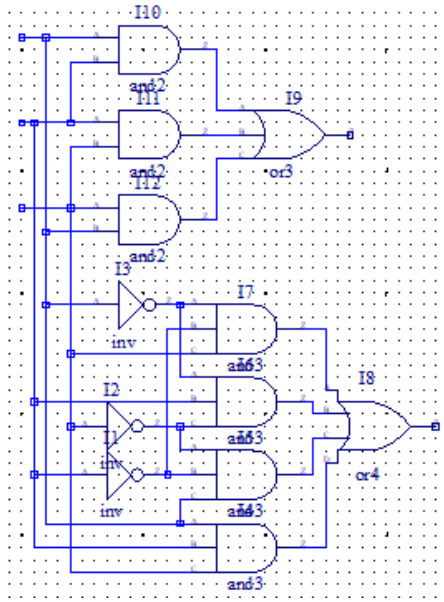


Figure 17: Schematic with logic elements and interconnections

In this diagram, the inputs and outputs do not have names yet. We will first give names to the inputs and outputs. In the toolbar, left-click on the fifth icon from the top (wire name). The *Set Wire Name* tab opens, and in the third box (which is the only one that is active) we enter the name we want (Figure 18).

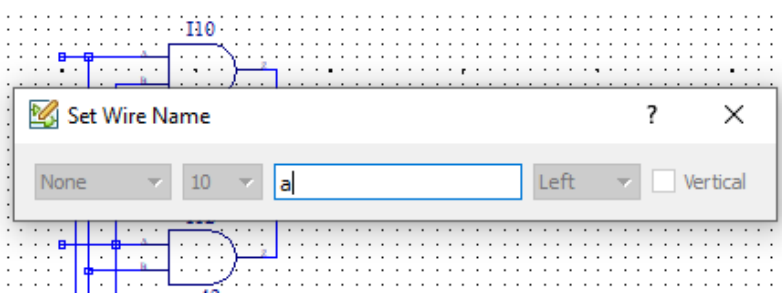
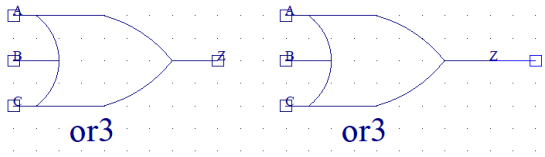


Figure 18: Naming inputs and outputs

Left-clicking on the square at the end of the wire will give this input the specified name (**a**). In the box above, write the name of the next input or output and left-click on the corresponding square. Repeat the procedure until each input and output has been given a name.

Before naming the outputs, take a look at Figure 19. On the left, you can clearly see that the inputs and outputs of each element have names: the inputs are labelled A, B, ... and the outputs are labelled Z. Therefore, you cannot give them another name (by left-clicking the corresponding square).



a) b)

Figure 19: Inputs and outputs of logic elements

In order to give names to the outputs of the circuit shown in Figure 17, short lines must be drawn at the ends (Z) of the outputs of the corresponding logic elements (Figure 19, b).

Next, you need to note where the inputs and outputs of the circuit are designed. Left-click on the seventh icon from the top in the toolbar (- Inputs/Outputs). The *Set IO Type* tab appears (Figure 20). In the first window of the card, click on the black triangle and select *Input*. Mouse over the inputs you want to select (it is possible to select one or several inputs at once). The inputs are bypassed by pressing the left mouse button.

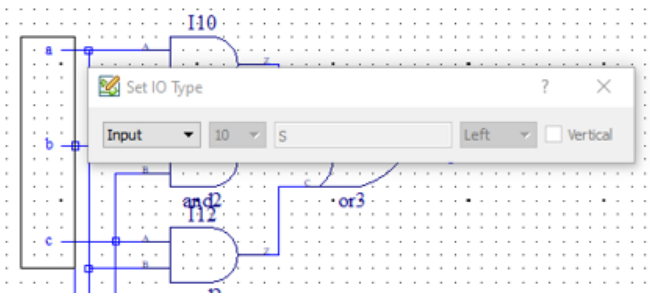


Figure 20: Labelling of the inputs to the scheme

In Figure 20, we can see that three inputs are bypassed. When you release the mouse button, outputs **a**, **b** and **c** are marked as inputs. Repeat this procedure until all inputs are marked.

The outputs must then be marked in the same way. This is done by selecting *Output* in the first box of the card.

The designed scheme must be saved. Left-click **File→Save <filename>** or press <Crtl> + <S>.

The designed scheme is shown in Figure 21.

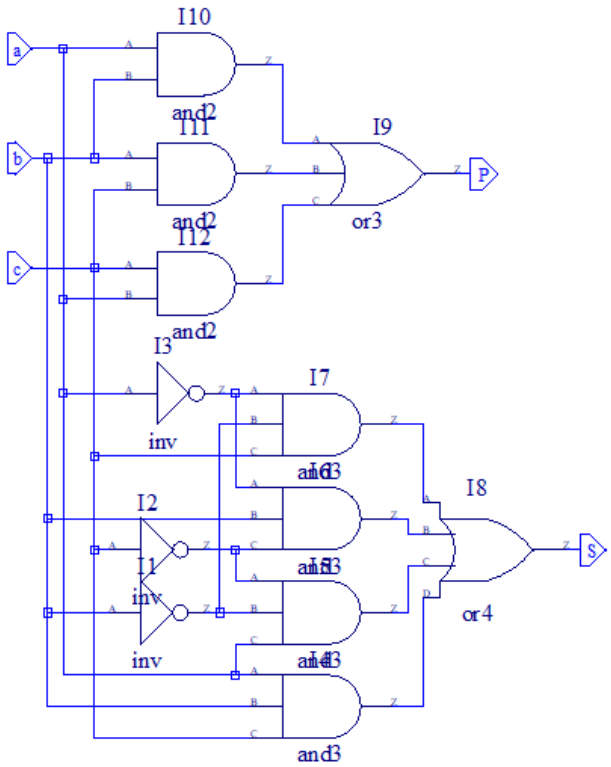


Figure 21: Designed scheme

Before invoking the simulation wizard, the graphical representation of the scheme needs to be translated into its VHDL signature. At the bottom of the left tab of the window, select ***Process***. The window that opens is shown in Figure 22.

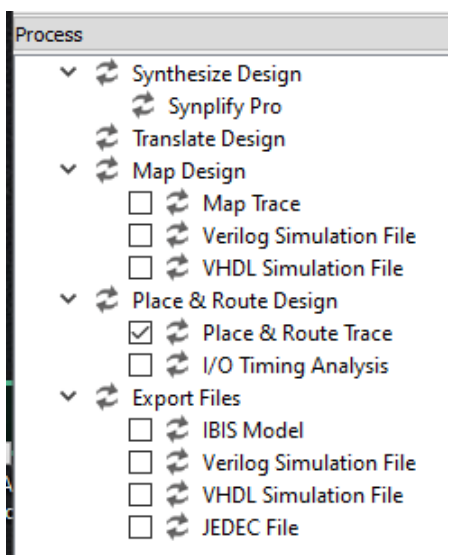


Figure 22: *Process* window

Left-click ***Synthesize Design***. Steps marked ***Synthesize Design*** and ***Synplify Pro*** are performed automatically. Then click on ***Translate Design***. If the design is error-free (this does not mean it is functionally correct), a green tick will appear next to ***Translate Design:***

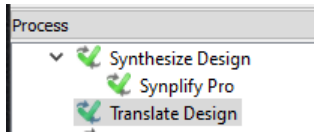


Figure 23: Successful truncation of the scheme

3. Developing a modelling project

A simulation project is created to model the scheme. To do this, select **Tools→Simulation Wizard** from the main menu or click on from the toolbar. The simulation project name window, shown in Figure 24, will open.

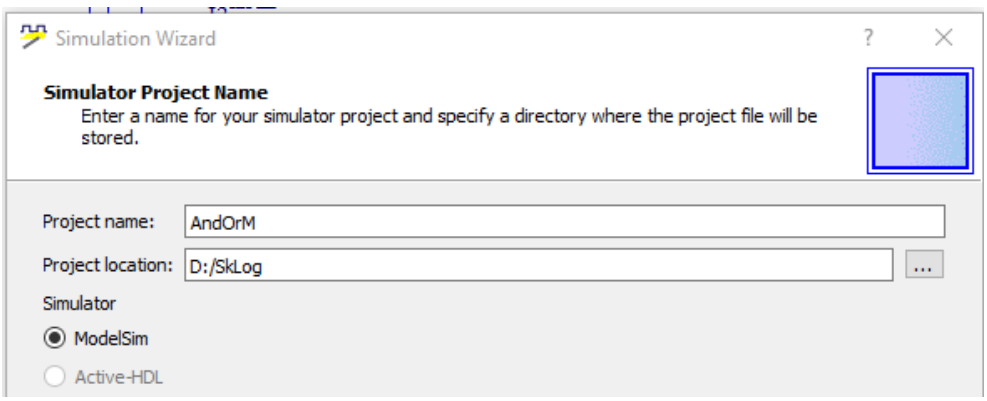


Figure 24. Modelling project name window

In the *Project name* box, enter the name of the modelling project and click **Next**. If RTL is disabled (greyed out) in the window (*Process Stage*) that appears, then the ***Translate Design*** steps have not been performed or errors have been detected during the translation. In this case, it is necessary to correct the errors, carry out the previous steps and repeat the creation of the modelling project.

You will need to press **Next** several times during the Simulation Wizard and **Finish** at the end. This will open the modelling window shown in Figure 25.

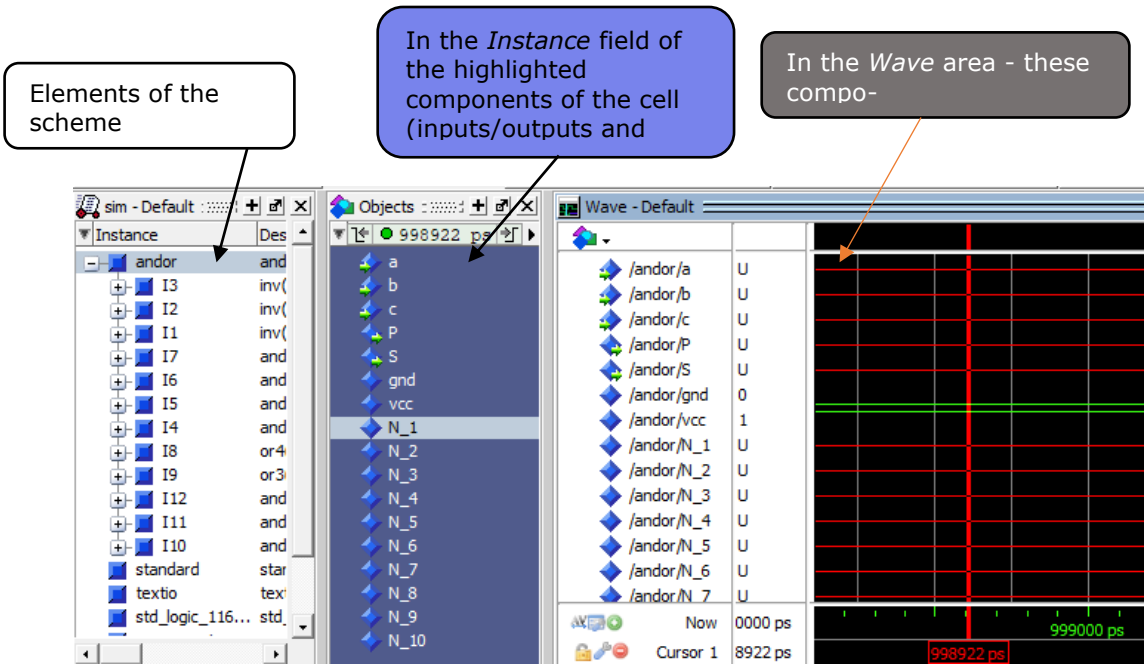


Figure 25. Modelling window

To check the correctness of the designed scheme (Figure 21), we will feed all possible combinations of values of the input variables into its inputs.

If we look at column **c of** the correctness table, the variation of the variable's values resembles a periodic sequence of tactical impulses. The same can be said for **a** and **b**:

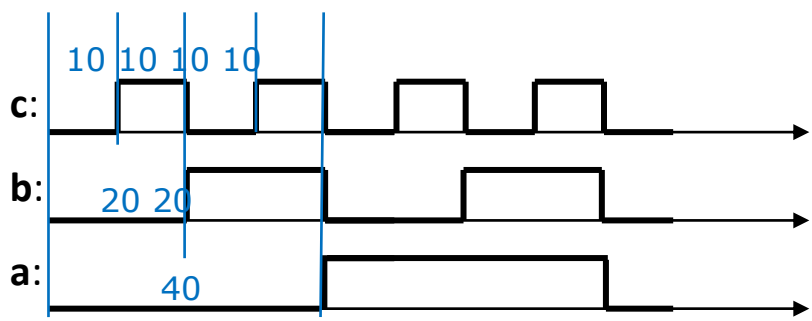
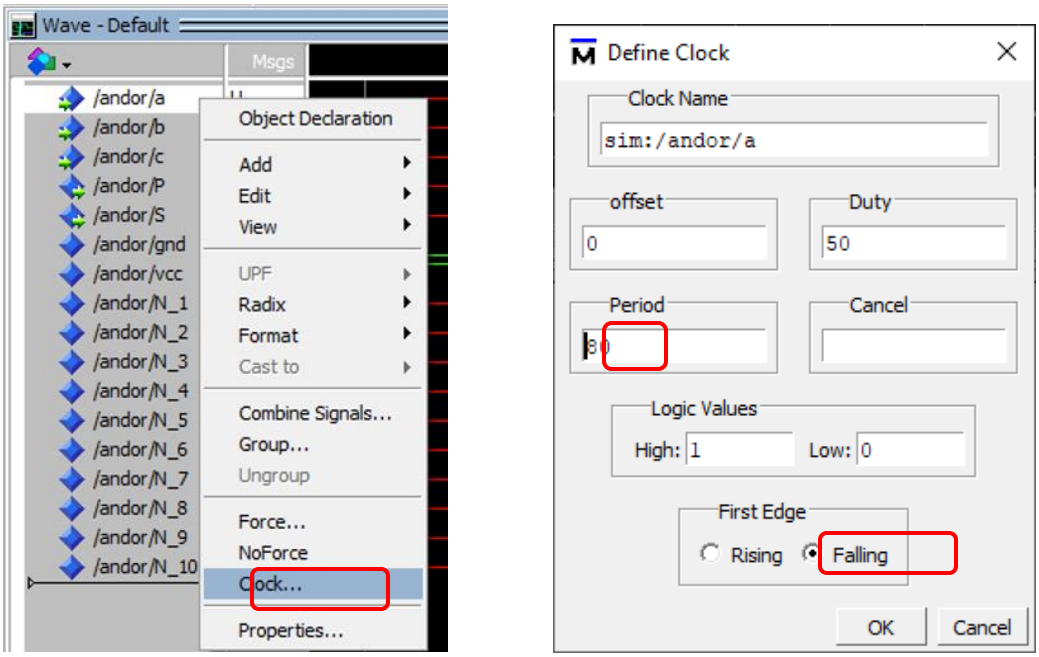


Figure 26: Changes in the values of the variables

This figure shows that input **c** is held for 10 picoseconds (of course, other durations can be set).

After selecting **a** in the *Wave* field, right-click and select ***Clock***... from the context menu. (Figure 27, a). In the *Define Clock* window that opens, we specify the period of variation **of a** to be 80 (Figure 27, b), and select ***Falling*** in the *First Edge* field to start the sequence at 0.



a) b)

Figure 27: Test stimulus generation

The following directive will appear in the *Transcript* area of the ModelSim window:

```
force -freeze sim:/andor/a 0 0, 1 {40 ps} -r 80
```

Sequences of values for variables **b** and **c** can be constructed in the same way.

Then specify the simulation time (80 ps) at the top of the window and click on the button on the right.

However, it is advisable to do otherwise. Look at the Directive as drafted (Figure 28). It contains the following elements:

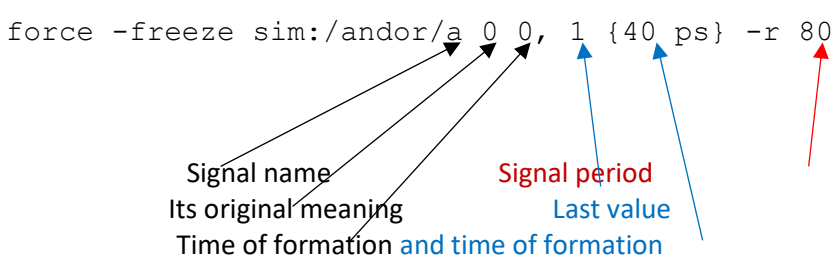


Figure 28: ***Clock*** directive

So let's copy this directive into the Notepad text editor and use it to create two more:

```
force -freeze sim:/andor/b 0 0, 1 {20 ps} -r 40
force -freeze sim:/andor/c 0 0, 1 {10 ps} -r 20
```

Before these directives, let's add a `restart` directive and a simulation run directive with a specified simulation run time:

```
restart -f
run 80
```

A set of directives

```
restart -f
force -freeze sim:/andor/a 0 0, 1 {40 ps} -r 80
force -freeze sim:/andor/b 0 0, 1 {20 ps} -r 40
force -freeze sim:/andor/c 0 0, 1 {10 ps} -r 20
run 80
```

let's write it to a file. This set of directives will be useful for iterative modelling of the operation of the scheme, as well as for other variants of the scheme.

After inserting the directives into the *Transcript* window under *VSIM xx*>, press <Enter>. The simulation results will then appear (Figure 29):

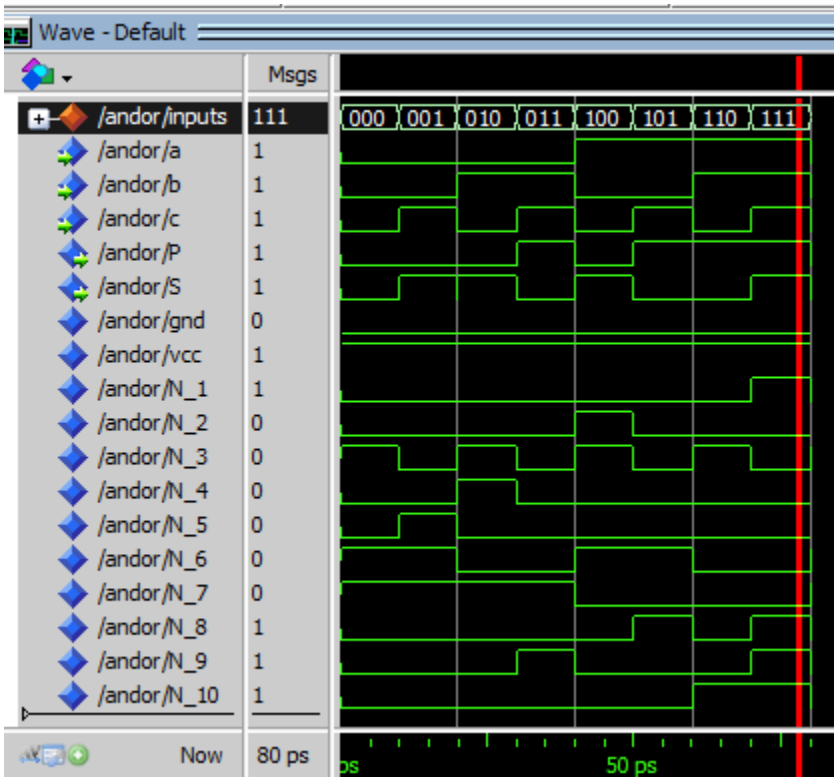
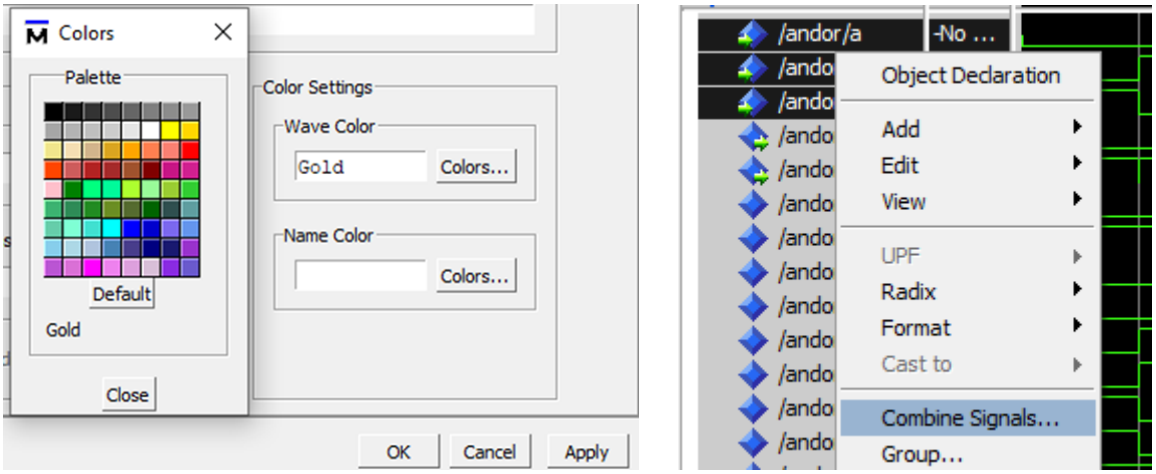


Figure 29. Simulation results

For more convenient analysis:

- change the colour of the output line by double-clicking on its name and selecting the colour in the *Wave Colors* box in the window that opens (Fig. 30, a); once the colour is selected, click **Close**, then **OK;**

- remove lines that are not needed - gnd, vcc, N_1 - N_10;
- Combine the input signals into a group, giving it a name and a value display system (here it will be "inputs" and *Radix* = ***Binary)***; to do this, select the signals to be combined and, by right-clicking, choose ***Combine Signals*** from the context menu (Figure 30, b).



a) b)

Figure 30: Colour selection and signal combining

We get the following picture:

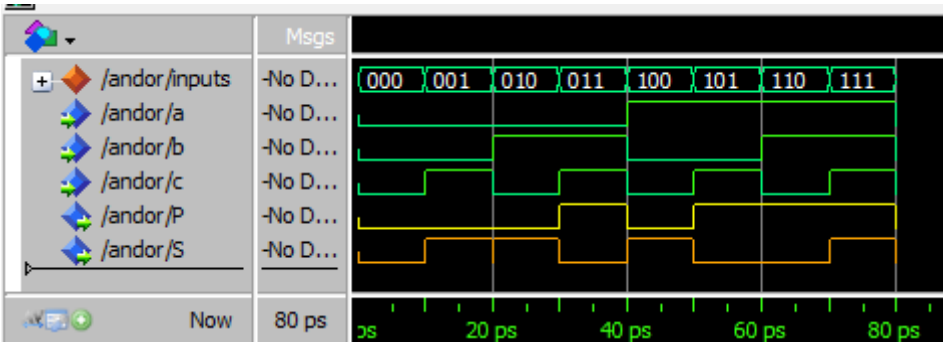


Figure 31: Managed simulation results

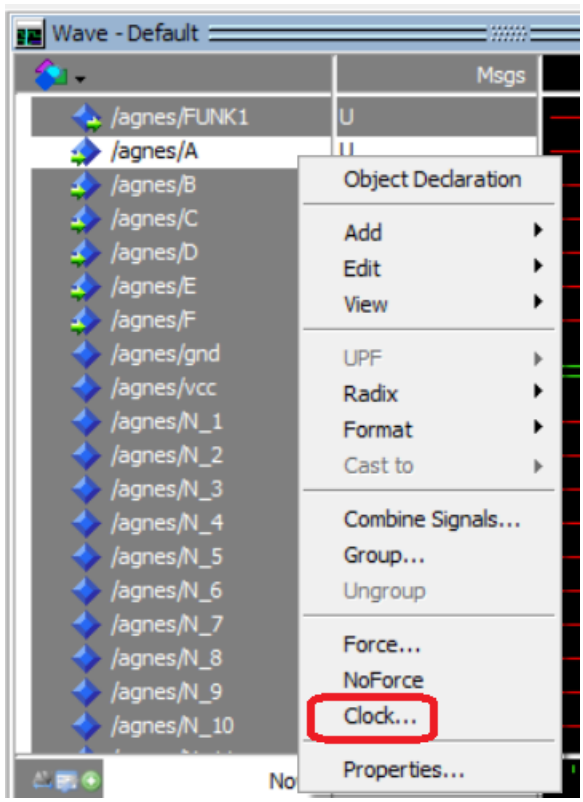
We can see that the carry (**P**) is formed when the input signals **<a**, **b**, **c>** are 001, 101, 110 and 111. The sum (**S**) is equal to 1 when the input signals **<a**, **b**, **c>** are 001, 010, 100 and 111.

PREPARATION OF TEST DATA FOR MODELSIM

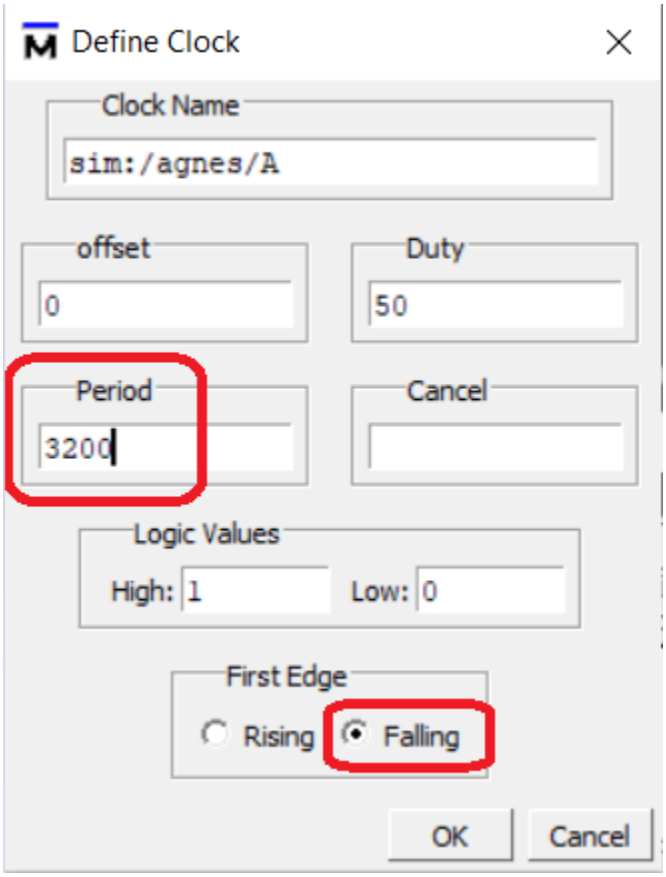
To simulate the operation of the (Lattice Diamond) circuits in ModelSim, we need to produce test data. We will make some suggestions that will make this work easier and save time.

1. *Preparation of test data for 1 laboratory paper*

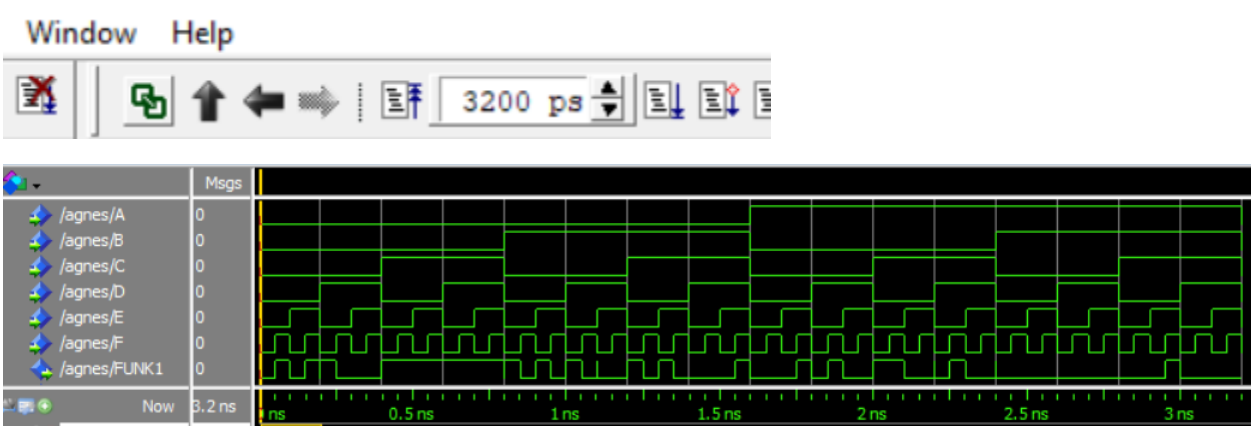
The first time we simulate the operation of a combinatorial scheme, as test data we have to construct a sequence of combinations of all possible values of the 6 variables (let's say they are named **A**, **B**, **C**, **D**, **E**, **F**): 000000, 000001, ..., 111110, 11111111. This can be achieved by selecting the variable in the ***Wave - Default window*** opened by ModelSim and selecting **Clock** from the context menu**:**



The Define Clock window will then open, where you will need to specify the period and initial value of the variable treated as a synchrosignal. Since variable **A** has the highest weight in the numeric form, it is given the maximum period (3200 in the figure); in the ***First Edge*** box, we tick **Falling** so that the variable starts with a value of 0 and then 1.



For variable **B,** you will then need to specify a period of twice the duration (1600); for the other variables, 800, 400, 200, 100 respectively. Then, by setting the simulation duration to 3200 ps in the top toolbar and clicking on the **Run** button next to it ( ), the simulation results will be displayed.



In the first 6 rows of the timeline, we can see 64 different combinations of values for the 6 variables (**A-F**), ranging from 000000 to 111111, and in the 7th row, the values obtained from the output of the flowchart (FUNK1).

2. *Developing testing directives*

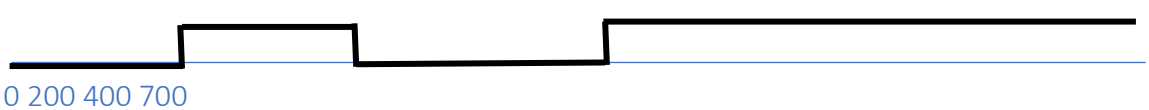
After the simulation with the variable values set in this way, we will find the following modeling directives in the lower part of the ModelSim window:

```
Transcript
force -freeze sim:/agnes/A 0 0, 1 {1600 ps} -r 3200
force -freeze sim:/agnes/B 0 0, 1 {800 ps} -r 1600
force -freeze sim:/agnes/C 0 0, 1 {400 ps} -r 800
force -freeze sim:/agnes/D 0 0, 1 {200 ps} -r 400
force -freeze sim:/agnes/E 0 0, 1 {100 ps} -r 200
force -freeze sim:/agnes/F 0 0, 1 {50 ps} -r 100
VSIM 9> run
```

We advise you to save these modelling directives in a text file. Then the next time you want to model the circuit, you will not need to specify the periods and initial values of the variables in the **Define Clock** window. Just insert the copied values after the last `VSIM 10>` and add the ***run 3200*** directive after it.

We will now show how to prepare similar directives corresponding to the required form of input signal **A**.

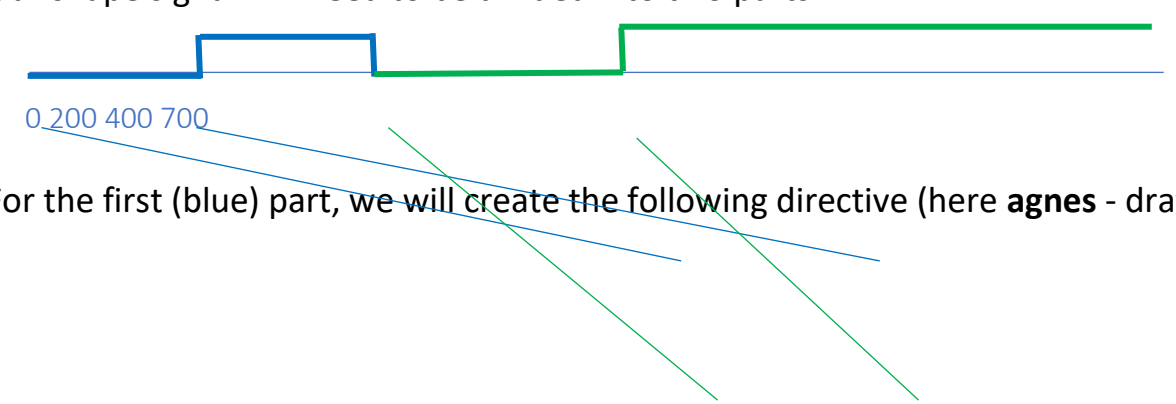
Let us need to form directives that correspond to this form of signal:



We will use directives in the following form:

force -freeze sim:/<Project>/<Signal> <Initial Moment> <Initial Value>, <Later Value> {<Later Value Start>}

Our shape signal will need to be divided into two parts:



For the first (blue) part, we will create the following directive (here **agnes** - draft):

```
force -freeze sim:/agnes/A 0 0, 1 {200 ps}
```

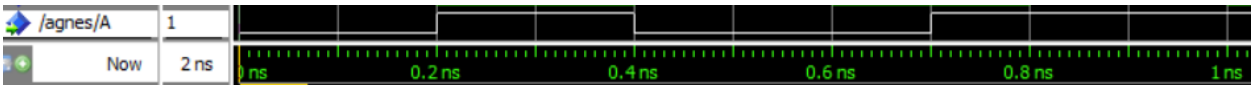
For the second (green) part, we will develop the following directive:

```
force -freeze sim:/agnes/A 0 400, 1 {700 ps}
```

Using the directives:

```
force -freeze sim:/agnes/A 0 0, 1 {200 ps}
force -freeze sim:/agnes/A 0 400, 1 {700 ps}
```

we got:



These two directives can be combined into one. In this case, the syntax of the Directive is slightly different:

force -freeze sim:/<Project>/<Signal> <Initial Moment> <Initial Value>, <Other Value> <Start>, <Other Value> <Start>, ...

When combined, we get the following directive:

```
force -freeze sim:/agnes/A 0 0, 1 200, 0 400, 1 700
```

Please note that the figure brackets and "ps" are not required.

UPLOADING DESIGNED DEVICES TO PLIS

LATTICE BREVIA STAND

A Field Programmable Gate Array (*FPGA*) is a ready-to-use integrated circuit that can be programmed by the user to perform the functions they require. You can read more about this in the **PLIS Programming** document on Moodle (https://moodle.ktu.edu/mod/resource/view.php?id=90865). As we will be using a **Lattice Brevia** bench in the labs, the following will focus on this type of bench.

The stand is small - a 76 x 76 mm plate (Figure 1):

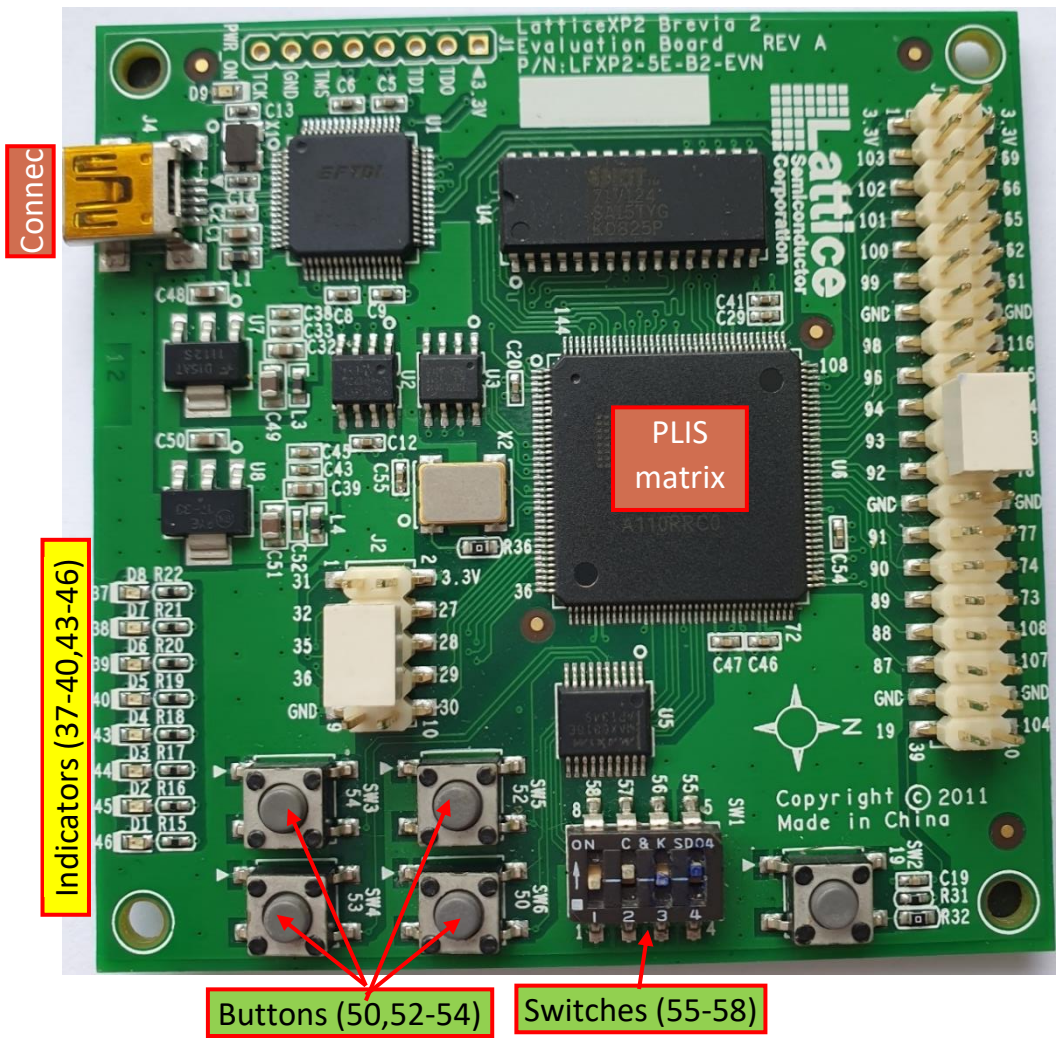


Figure 1. Lattice Brevia stand

This figure highlights the following key elements:

- ***PLIS matrix*** to which the designed schematic is uploaded from the PC, using the USB cable supplied with the stand, connected via a ***connector***;
- 8 ***Indicators*** - LEDs that light up when a low level signal is applied;
- 4 ***Switches*** with 2 positions - ON (1) - on (tiny lever pushed up (in the case oriented in the figure), 0 - off (tiny lever pushed down);
- ***Buttons*** that when pressed produce 0 and when released produce 1 if they are set to PULLMODE = UP in the **Spreadsheet View** table.

Indicators, buttons and switches have their own numbers (Figure 2) which link them to specific inputs and outputs (signals) in the **Spreadsheet View** table.

Įrenginys	Prievado pavadinimas (Pin name)	Papildomos nuostatos
Mygtukas 1	50	PULLMODE - UP
Mygtukas 2	52	PULLMODE - UP
Mygtukas 3	53	PULLMODE - UP
Mygtukas 4	54	PULLMODE - UP
LED 1	37	PULLMODE - UP
LED 2	38	PULLMODE - UP
LED 3	39	PULLMODE - UP
LED 4	40	PULLMODE - UP
LED 5	43	PULLMODE - UP
LED 6	44	PULLMODE - UP
LED 7	45	PULLMODE - UP
LED 8	46	PULLMODE - UP
Jungiklis 1	55	
Jungiklis 2	56	
Jungiklis 3	57	
Jungiklis 4	58	

Figure 2: Lattice Brevia stand item numbers

PREPARATION OF THE SCHEME FOR UPLOADING TO PLIS

We will demonstrate this by taking as an example the 4-column universal impulse register we have designed (Figure 3).

To simulate the operation of this register, we have combined its signals (*Combine Signals*) into the following groups:

- ***Duom*** - bits of information to be recorded (**B3-B0**);
- ***Oper*** - bits (**A1-A0)** selecting the operation to be performed;
- ***Register*** - register output (**Q3-Q0**);

Using the modelling directives:

```
restart -f
force -freeze sim:/Univers/R 1 0, 0 {5 ps}
force -freeze sim:/Univers/C 0 0, 1 {25 ps} -r 50
force -freeze sim:/Univers/DR 0 0, 1 {200 ps}
force -freeze sim:/Univers/DR 0 300
force -freeze sim:/Univers/DL 0 0, 1 {550 ps}
force -freeze sim:/Univers/Duom 1001 0
force -freeze sim:/Univers/Oper 11 0, 00 100, 01 150, 00 350, 11 400, 10 450
run 700
```

we obtained the timing diagram of the boost register in Figure 4.

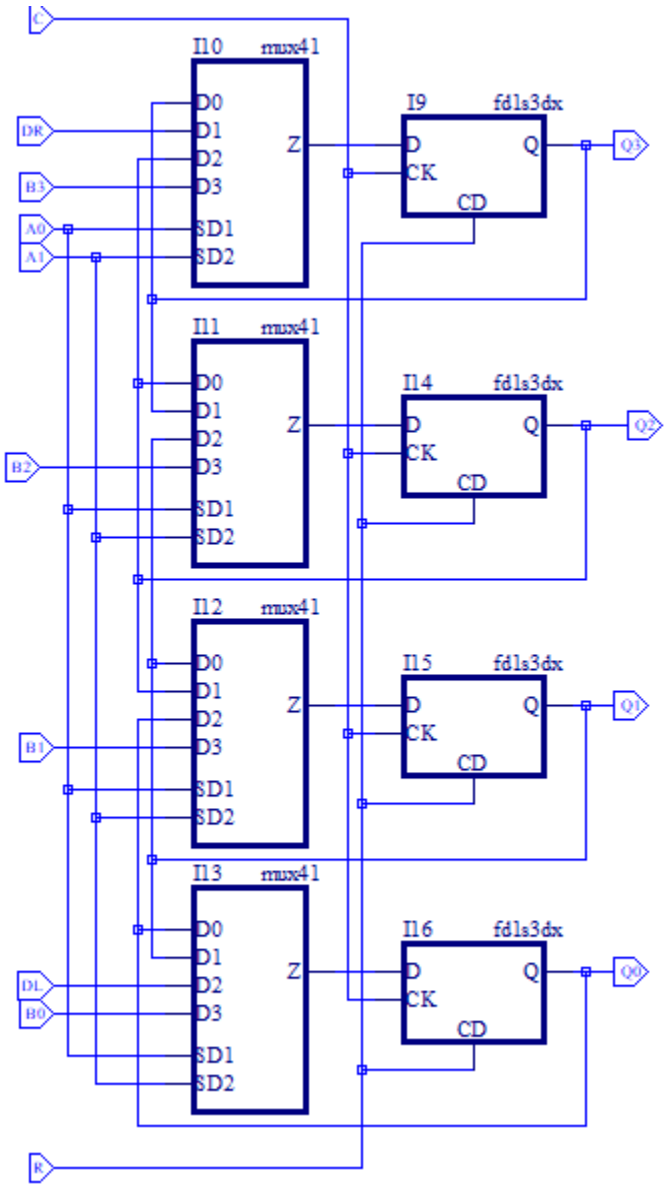


Figure 3: Universal boost register

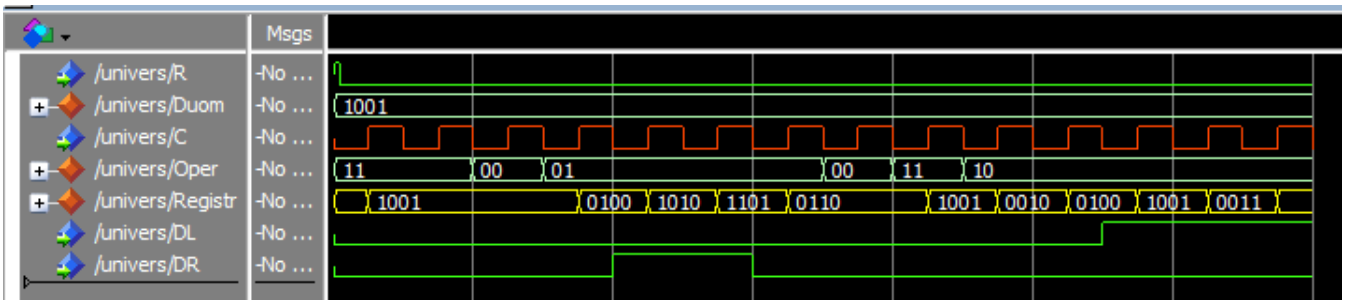


Figure 4: Timing diagram for the universal boost register

When we prepare the impetus register scheme for uploading to PLIS, we need to modify it:

- The inverters are inserted in the outputs of the triggers (shown below with a red dotted box);
- Inputs **B3-B0** are replaced by logic 0 and logic 1 oscillators (vlo and vhi);
- If the triggers are set to zero by a short single-ended signal, an inverter is added to this circuit (Figure 5):

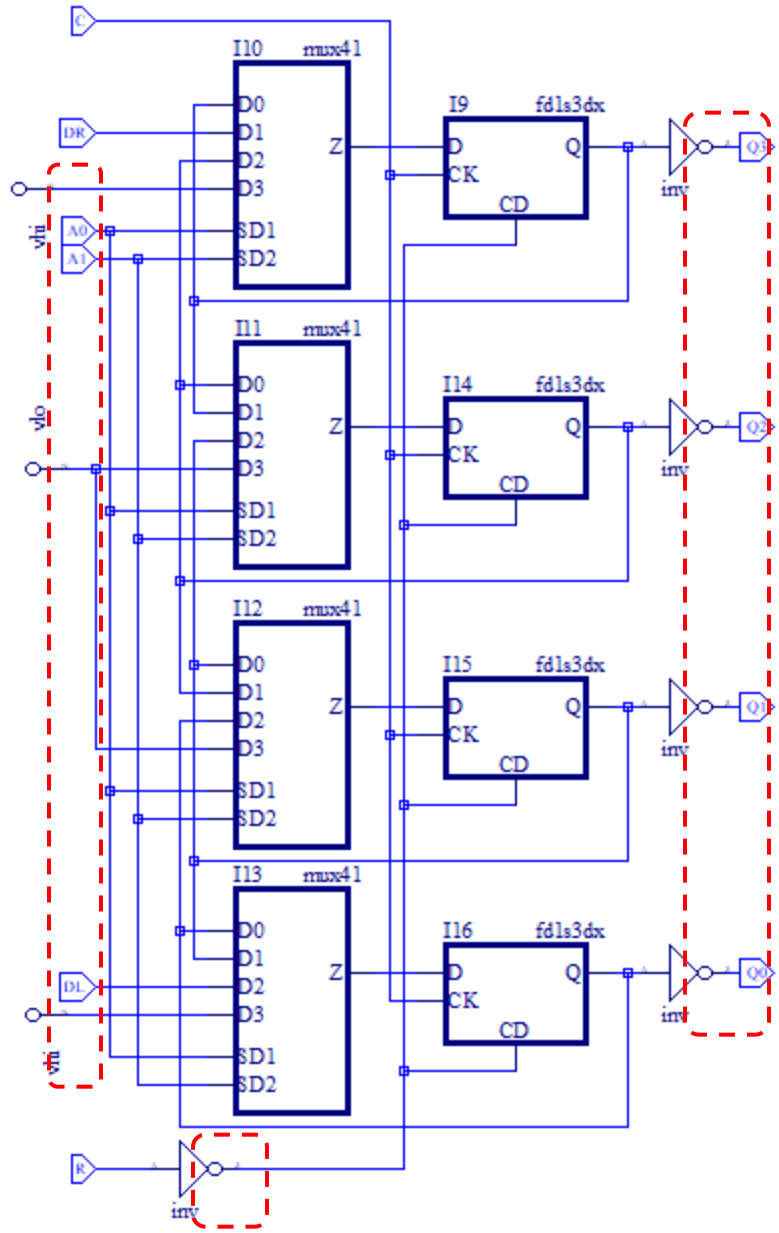


Figure 5: Schematic of the Universal Displacement Register ready for upload to PLIS

When replacing the inputs **B3-B0 with** the logic 0 and logic 1 generators, we need to think carefully about the word to be written to the register, so that we can demonstrate the operation of the register by evaluating the peculiarities of its operations:

1. For the universal boost register, it is recommended that the number of 0's and 1's in a word be approximately equal.
2. In the case of a specialised boost register, the following points need to be considered:

- If there are 2 columns between the strokes, do not use the word 010101 as the result of the stroke will be similar to the original word;
- For the arithmetic boost, it is advisable to use a negative number code, so vhi is recommended for the top column **Bn**.

1. Setting parameters and uploading the scheme to PLIS

Before programming this stand, it is necessary to specify the device type ***LatticeXP2-5E-6TN144C***. To do this, select **Project** → **Device** from the main menu in Lattice Diamond and specify the following parameters in the window that opens:

- Family - LatticeXP2,
- Device - LFXP2-5E,
- Package type - TQFP144.

After the modified schematic has been translated, select **Tools** → **Spreadsheet View** from the main menu**.** The following window will open (Figure 6):

	Name	Group By	Pin	BANK	BANK_VCC	IO_TYPE	PULLMODE
1	All Ports	N/A	N/A	N/A	N/A		
1.1	Input	N/A	N/A	N/A	N/A	N/A	N/A
1.1.1	A0	N/A	B1	3	Auto	LVCMOS25	UP
1.1.2	A1	N/A	C3	3	Auto	LVCMOS25	UP
1.1.3	Clock	N/A	N/A	N/A	N/A	N/A	N/A
1.1.3.1	C	N/A	N3	2	Auto	LVCMOS25	DOWN
1.1.4	DL	N/A			N/A	LVCMOS25	DOWN
1.1.5	DR	N/A			N/A	LVCMOS25	DOWN
1.1.6	R	N/A			N/A	LVCMOS25	DOWN
1.2	Output	N/A	N/A	N/A	N/A	N/A	N/A
1.2.1	Q0	N/A	D1	3	Auto	LVCMOS25	DOWN
1.2.2	Q1	N/A	B8	0	Auto	LVCMOS25	DOWN
1.2.3	Q2	N/A	C8	0	Auto	LVCMOS25	DOWN
1.2.4	Q3	N/A	B13	0	Auto	LVCMOS25	DOWN

Figure 6: Spreadsheet View window

For the necessary contacts connected to the buttons and LEDs, the PULLMODE parameter must be set to UP to generate a low logic level when the button is pressed.

We assign switches to form the retrieval signals (**A1** and **A0**) because the same impulse will have to be repeated several times.

We also include switches to form **DR** and **DL** signals.

We assign buttons for the reset (**R**) and synchronisation (**C**) signals.

We assign indicators to monitor the register outputs (**Q3-Q0**).

This gives us the following picture (Figure 7):

	Name	Group By	Pin	BANK	VREF	IO_TYPE	PULLMODE
1	All Ports	N/A	N/A	N/A	N/A		
1.1	Input	N/A	N/A	N/A	N/A	N/A	N/A
1.1.1	A0	N/A	55	5	N/A	LVCMOS25	UP
1.1.2	A1	N/A	56	5	N/A	LVCMOS25	UP
1.1.3	Clock	N/A	N/A	N/A	N/A	N/A	N/A
1.1.3.1	C	N/A	50	5	N/A	LVCMOS25	UP
1.1.4	DL	N/A	57	5	N/A	LVCMOS25	UP
1.1.5	DR	N/A	58	5	N/A	LVCMOS25	UP
1.1.6	R	N/A	54	5	N/A	LVCMOS25	UP
1.2	Output	N/A	N/A	N/A	N/A	N/A	N/A
1.2.1	Q0	N/A	40	5	N/A	LVCMOS25	UP
1.2.2	Q1	N/A	39	5	N/A	LVCMOS25	UP
1.2.3	Q2	N/A	38	5	N/A	LVCMOS25	UP
1.2.4	Q3	N/A	37	5	N/A	LVCMOS25	UP

Figure 7: Completed Spreadsheet View window

After saving the changes in the ***Spreadsheet View*** window, a project constraints *.lpf file is automatically created.

The physical positioning (planning) of the scheme on the board is done via the ***Place and Route*** menu item. Perform the **Place and Route** step from the **Process** menu.

After the ***Place and Route*** step it is necessary to generate the FPGA programming file. To do this, tick the ***JEDEC File checkbox*** in the ***Export Files*** section of the ***Process*** window. By double-clicking **Export Files**, the file *project_name.jed* will be exported to the appropriate folder (Figure 8):

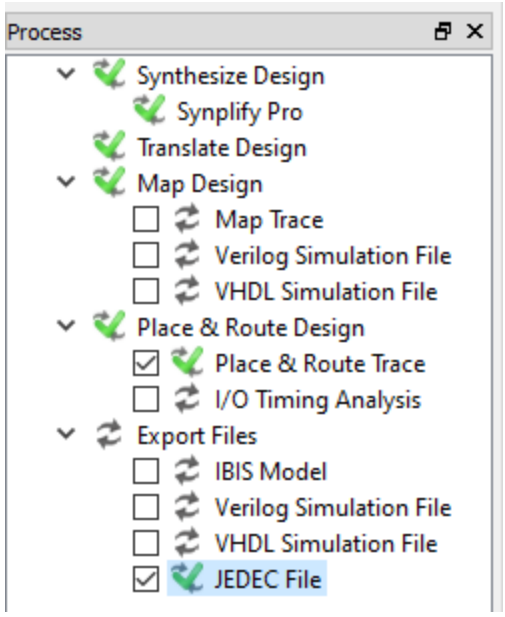
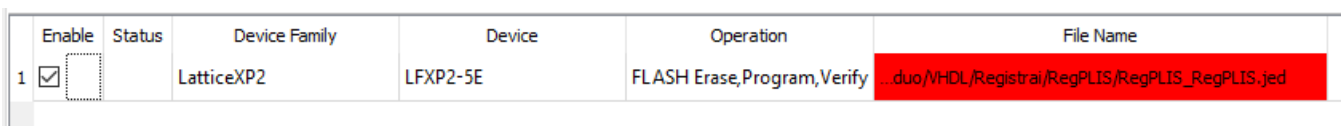


Figure 8: Process menu after programming readiness

This file will be used to program the matrix.

Then connect the stand to the USB port and select Tools→ **Programmer** from the main menu. In the programmer window that appears, check that the ***Device Family and Device*** match the available matrix (in this lab bench it is LatticeXP2-5E-6TN144C; if it does not match, it means that the wrong library was selected during the creation of the project). In addition, make sure that the ***File Name*** column specifies the required (for this project) *.jed file.

Sometimes the application indicates that it was unable to find the file (area in red) (Figure 9):



	Enable	Status	Device Family	Device	Operation	File Name
1	☑		LatticeXP2	LFXP2-5E	FLASH Erase,Program,Verify	...duo/VHDL/Registrai/RegPLIS/RegPLIS_RegPLIS.jed

Figure 9: Programmer information

You then have to search for this file in folders (Figure 10):

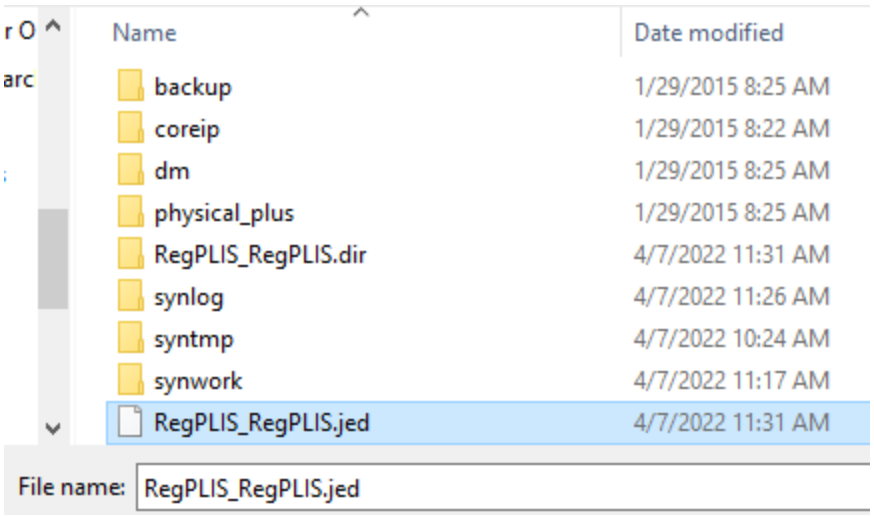


Figure 10. Searching for a .jed file

Once it is found, you need to initiate PLIS programming by selecting **Design** → **Program** from the main menu**.**

The first programming attempt may fail, in which case ***FAIL*** appears in the ***Status*** column ***(***Figure 11):

Figure 11: Programming error

In this case, click on the **Detect Cable** button on the right **(**Figure 12):

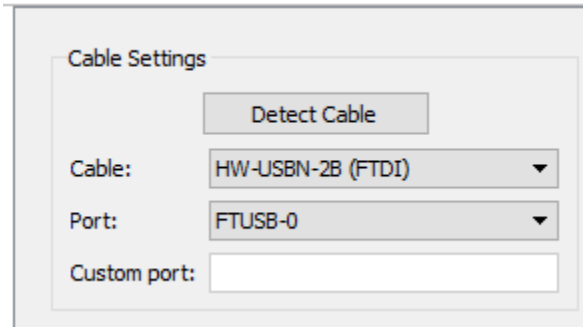


Figure 12: Searching for a connection cable

and select another cable (Figure 13):

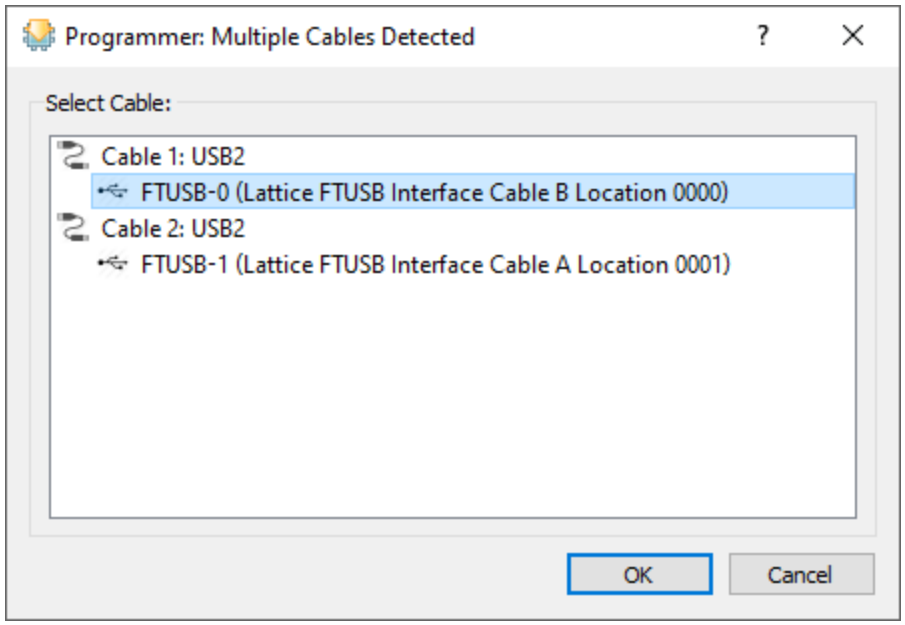


Figure 13: Selecting the connection cable

This usually helps, and after initiating PLIS programming, ***PASS*** appears in the ***Status*** column (Figure 14):

	Enable	Status	Device Family	Device
1	☑	PASS	LatticeXP2	LFXP2-5E

Figure 14: Programming succeeded

Now all you need to do is control the operation of the stand using switches and buttons.

Here is how the image (Figure 15) looks when the word 1001 has been written to the register:

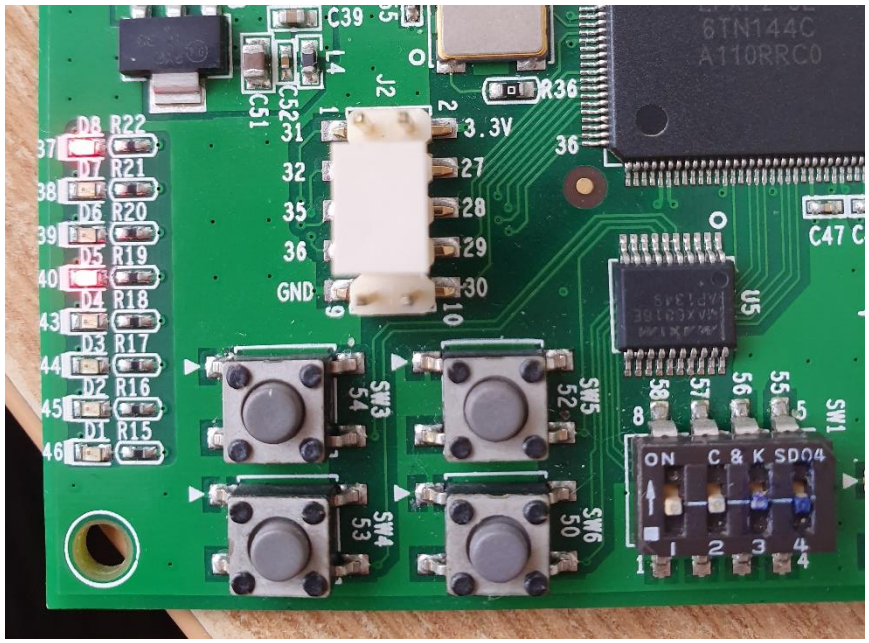


Figure 15: Indication of a recorded word (37-40 LEDs)

2. Preparation of the VHDL inventory for uploading to PLIS

To load a device into PLIS that is described in VHDL, you need to modify the description slightly. Since the LEDs used for the indication light up when a low level signal (0) is applied to them, the outputs of the circuit need to be inverted. For example, for a decimal counter, instead of Q <= std_logic_vector(Q30); it should be Q <= **not** std_logic_vector(Q30);